

Que 1:

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 102 // number of cycles

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int turn = 0; // 0 -> A, 1 -> B, 2 -> C

void *printA(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        while (turn != 0) {
            pthread_cond_wait(&cond, &lock);
        }
        printf("A ");
        fflush(stdout);
        turn = 1;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void *printB(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        while (turn != 1) {
            pthread_cond_wait(&cond, &lock);
        }
        printf("B ");
        fflush(stdout);
        turn = 2;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void *printC(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        while (turn != 2) {
            pthread_cond_wait(&cond, &lock);
        }
    }
}
```

```

    printf("C\n");
    fflush(stdout);
    turn = 0;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&lock);
}
return NULL;
}

int main() {
    pthread_t tA, tB, tC;
    pthread_create(&tA, NULL, printA, NULL);
    pthread_create(&tB, NULL, printB, NULL);
    pthread_create(&tC, NULL, printC, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);
    pthread_join(tC, NULL);
    return 0;
}

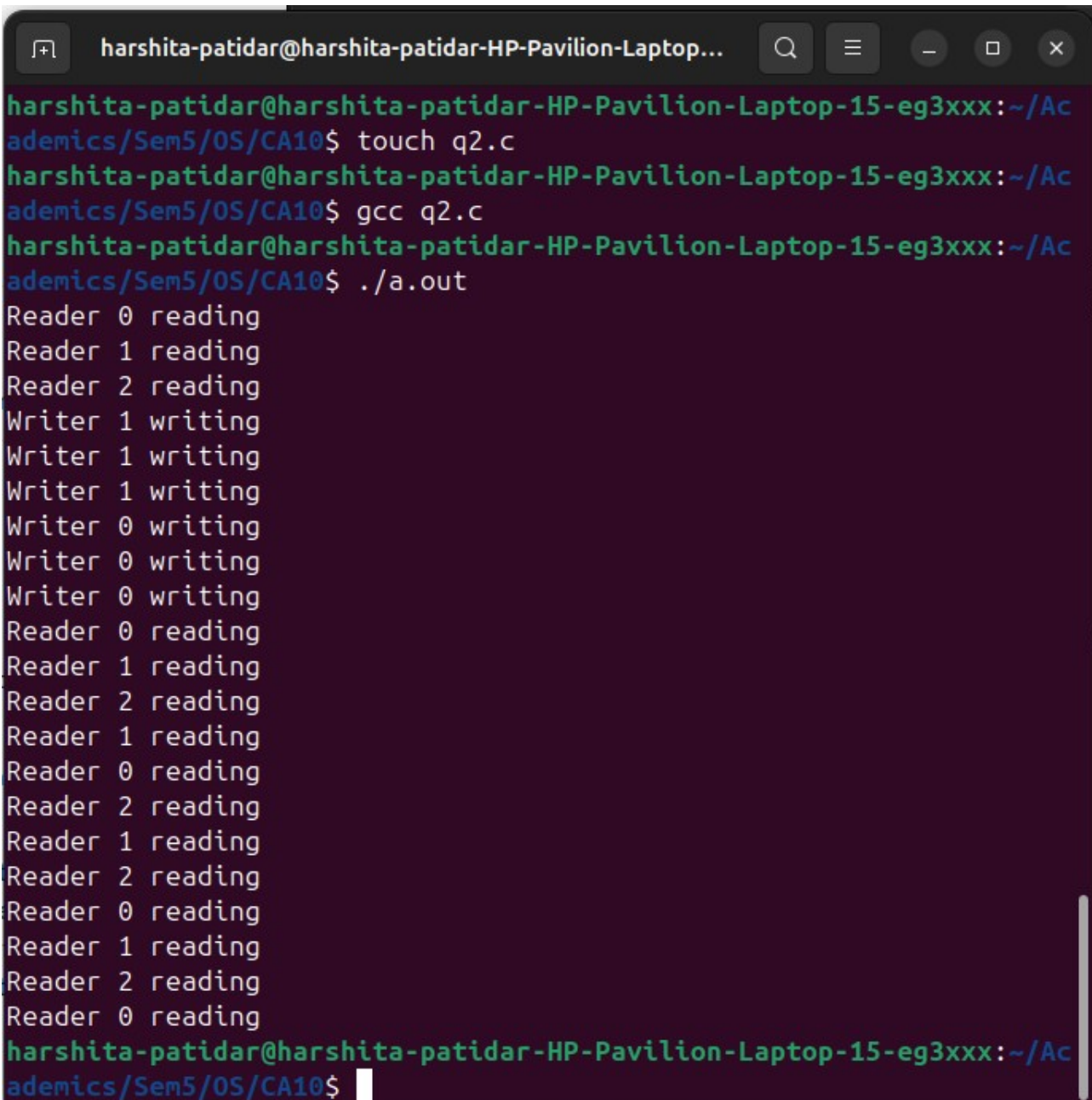
```

Screenshot:

[illegible]

Observation:

Using mutex and condition variables ensures strict ordered thread execution ($A \rightarrow B \rightarrow C$) without race conditions or random interleaving.

Que 2:**Screenshot:**

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$ touch q2.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$ gcc q2.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$ ./a.out
Reader 0 reading
Reader 1 reading
Reader 2 reading
Writer 1 writing
Writer 1 writing
Writer 1 writing
Writer 0 writing
Writer 0 writing
Writer 0 writing
Reader 0 reading
Reader 1 reading
Reader 2 reading
Reader 1 reading
Reader 0 reading
Reader 2 reading
Reader 1 reading
Reader 2 reading
Reader 0 reading
Reader 1 reading
Reader 2 reading
Reader 0 reading
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$
```

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t can_read = PTHREAD_COND_INITIALIZER;
pthread_cond_t can_write = PTHREAD_COND_INITIALIZER;

int read_count = 0;
int write_count = 0;
int waiting_writers = 0;

void start_read() {
    pthread_mutex_lock(&lock);
    while (write_count == 1 || waiting_writers > 0) {
        pthread_cond_wait(&can_read, &lock);
    }
    read_count++;
    pthread_mutex_unlock(&lock);
}

void end_read() {
    pthread_mutex_lock(&lock);
    read_count--;
    if (read_count == 0)
        pthread_cond_signal(&can_write);
    pthread_mutex_unlock(&lock);
}

void start_write() {
    pthread_mutex_lock(&lock);
    waiting_writers++;
    while (read_count > 0 || write_count == 1) {
        pthread_cond_wait(&can_write, &lock);
    }
    waiting_writers--;
    write_count = 1;
    pthread_mutex_unlock(&lock);
}

void end_write() {
    pthread_mutex_lock(&lock);
    write_count = 0;

    if (waiting_writers > 0)
        pthread_cond_signal(&can_write);
    else
        pthread_cond_broadcast(&can_read);

    pthread_mutex_unlock(&lock);
}
```

```

}

void *reader(void *id) {
    for (int i = 0; i < 5; i++) {
        start_read();
        printf("Reader %ld reading\n", (long)id);
        usleep(100000);
        end_read();
    }
    return NULL;
}

void *writer(void *id) {
    for (int i = 0; i < 3; i++) {
        start_write();
        printf("Writer %ld writing\n", (long)id);
        usleep(150000);
        end_write();
    }
    return NULL;
}

int main() {
    pthread_t r[3], w[2];
    for (long i = 0; i < 3; i++) pthread_create(&r[i], NULL, reader, (void *)i);
    for (long i = 0; i < 2; i++) pthread_create(&w[i], NULL, writer, (void *)i);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    return 0;
}

```

Observation:

Adding writer-priority using condition variables prevents writer starvation and ensures fair access between readers and writers.

Que 3:

Code:

```

#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_LOGS 10

char *log_buffer[MAX_LOGS];
int count = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

```

```

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void *worker(void *id) {
    for (int i = 0; i < 3; i++) {
        char msg[64];
        sprintf(msg, "Worker %ld message %d", (long)id, i);

        pthread_mutex_lock(&lock);
        while (count == MAX_LOGS) {
            pthread_cond_wait(&cond, &lock);
        }

        log_buffer[count++] = strdup(msg);
        printf("Worker %ld queued log. (count=%d)\n", (long)id, count);

        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&lock);

        usleep(100000);
    }
    return NULL;
}

void *logger(void *arg) {
    FILE *f = fopen("log.txt", "w");
    if (!f) {
        perror("fopen");
        return NULL;
    }

    while (1) {
        pthread_mutex_lock(&lock);
        while (count == 0) {
            pthread_cond_wait(&cond, &lock);
        }

        char *msg = log_buffer[--count];
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&lock);

        fprintf(f, "%s\n", msg);
        fflush(f);
        printf("Logger wrote: %s\n", msg);
        free(msg);

        usleep(50000);
    }

    fclose(f);
    return NULL;
}

```

```

int main() {
    pthread_t log_thread, workers[3];
    pthread_create(&log_thread, NULL, logger, NULL);
    for (long i = 0; i < 3; i++)
        pthread_create(&workers[i], NULL, worker, (void *)i);

    for (int i = 0; i < 3; i++)
        pthread_join(workers[i], NULL);

    sleep(1);

    pthread_cancel(log_thread);
    pthread_join(log_thread, NULL);

    printf("\nAll workers finished. Check 'log.txt' for output.\n");
    return 0;
}

```

Screenshot:

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Acad...
OS/CA10$ gcc Q3_12340920.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/
OS/CA10$ ./a.out
Worker 0 queued log. (count=1)
Worker 1 queued log. (count=2)
Logger wrote: Worker 1 message 0
Worker 2 queued log. (count=2)
Logger wrote: Worker 2 message 0
Worker 0 queued log. (count=2)
Worker 1 queued log. (count=3)
Worker 2 queued log. (count=4)
Logger wrote: Worker 2 message 1
Logger wrote: Worker 1 message 1
Worker 0 queued log. (count=3)
Worker 1 queued log. (count=4)
Worker 2 queued log. (count=5)
Logger wrote: Worker 2 message 2
Logger wrote: Worker 1 message 2
Logger wrote: Worker 0 message 2
Logger wrote: Worker 0 message 1
Logger wrote: Worker 0 message 0

All workers finished. Check 'log.txt' for output.
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/

```

Observation:

Using mutex and condition variables eliminates busy-waiting and ensures no log messages are lost in the thread-safe logger.

Que 4:**Code:**

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NTHREADS 3

struct thread_event {
    pthread_mutex_t m;
    pthread_cond_t c;
    int awake;
};

struct thread_event events[NTHREADS];

void *sleeper(void *id) {
    long tid = (long)id;
    pthread_mutex_lock(&events[tid].m);

    while (!events[tid].awake) {
        printf("Thread %ld sleeping...\n", tid);
        pthread_cond_wait(&events[tid].c, &events[tid].m);
    }

    printf("Thread %ld woke up!\n", tid);

    pthread_mutex_unlock(&events[tid].m);
    return NULL;
}

void *waker(void *arg) {
    sleep(2);

    printf("Waker: waking all threads...\n");

    for (int i = 0; i < NTHREADS; i++) {
        pthread_mutex_lock(&events[i].m);
        events[i].awake = 1;
        pthread_cond_signal(&events[i].c);
        pthread_mutex_unlock(&events[i].m);
    }

    return NULL;
}
```



```

int main() {
    pthread_t t[NTHREADS], w;

    for (int i = 0; i < NTHREADS; i++) {
        events[i].awake = 0;
        pthread_mutex_init(&events[i].m, NULL);
        pthread_cond_init(&events[i].c, NULL);
        pthread_create(&t[i], NULL, sleeper, (void *) (long)i);
    }

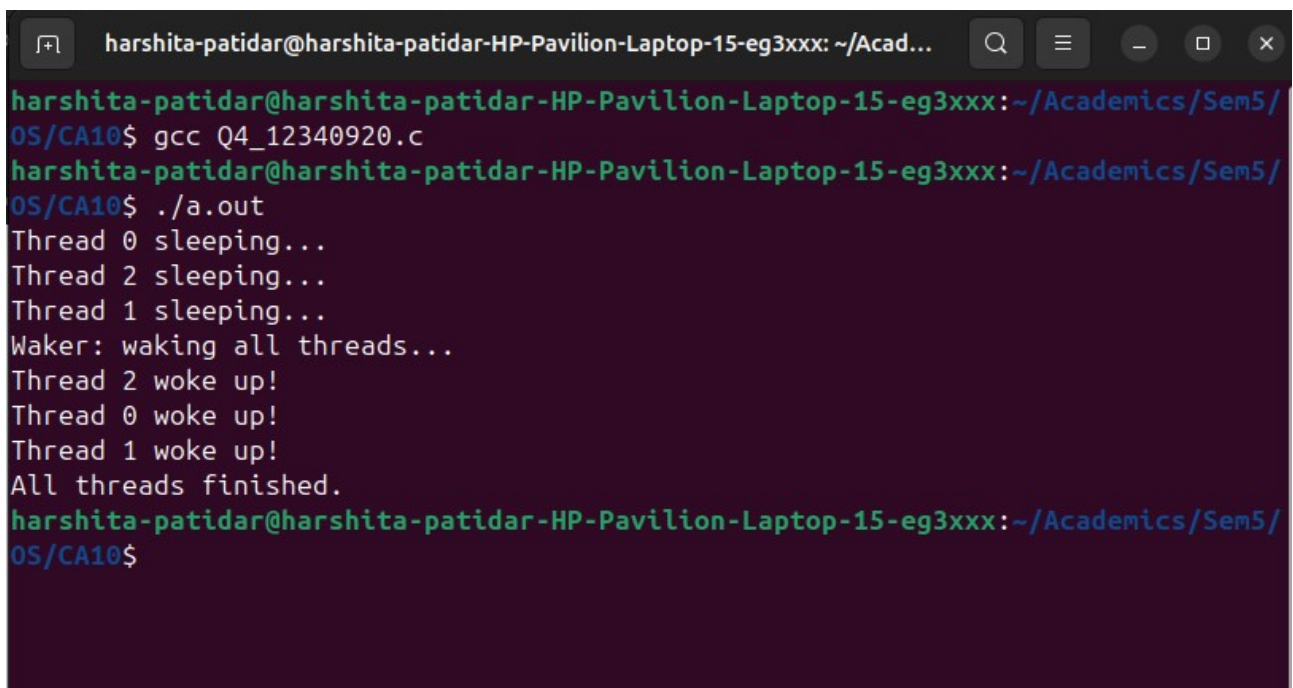
    pthread_create(&w, NULL, waker, NULL);
    pthread_join(w, NULL);

    for (int i = 0; i < NTHREADS; i++)
        pthread_join(t[i], NULL);

    printf("All threads finished.\n");
    return 0;
}

```

Screenshot:



```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Acad...
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$ gcc Q4_12340920.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$ ./a.out
Thread 0 sleeping...
Thread 2 sleeping...
Thread 1 sleeping...
Waker: waking all threads...
Thread 2 woke up!
Thread 0 woke up!
Thread 1 woke up!
All threads finished.
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA10$

```

Observation:

Using condition variables for sleep–wake synchronization prevents busy-waiting and ensures all sleeping threads are properly awakened.

Que 5:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_JOBS 5
#define NUM_WORKERS 3

int jobs[MAX_JOBS];
int count = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t job_available = PTHREAD_COND_INITIALIZER;
pthread_cond_t space_available = PTHREAD_COND_INITIALIZER;

void *dispatcher(void *arg) {
    int job_id = 1;
    while (job_id <= 10) {
        pthread_mutex_lock(&lock);

        while (count == MAX_JOBS)
            pthread_cond_wait(&space_available, &lock);

        jobs[count++] = job_id;
        printf("Dispatcher added job %d (count=%d)\n", job_id, count);

        pthread_cond_signal(&job_available);
        pthread_mutex_unlock(&lock);

        job_id++;
        usleep(100000);
    }
    return NULL;
}

void *worker(void *arg) {
    long id = (long)arg;

    while (1) {
        pthread_mutex_lock(&lock);

        while (count == 0)
            pthread_cond_wait(&job_available, &lock);

        int job = jobs[--count];
        printf("Worker %ld processing job %d (remaining=%d)\n", id, job, count);

        pthread_cond_signal(&space_available);
        pthread_mutex_unlock(&lock);
    }
}
```

```

        usleep(200000);
    }
    return NULL;
}

int main() {
    pthread_t disp, workers[NUM_WORKERS];

    pthread_create(&disp, NULL, dispatcher, NULL);
    for (long i = 0; i < NUM_WORKERS; i++)
        pthread_create(&workers[i], NULL, worker, (void *)i);

    pthread_join(disp, NULL);
    sleep(2);

    printf("All jobs dispatched. Exiting...\n");
    return 0;
}

```

Screenshot:

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/S...
10$ gcc Q5_12340920.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA
10$ ./a.out
Dispatcher added job 1 (count=1)
Worker 0 processing job 1 (remaining=0)
Dispatcher added job 2 (count=1)
Worker 1 processing job 2 (remaining=0)
Dispatcher added job 3 (count=1)
Worker 0 processing job 3 (remaining=0)
Dispatcher added job 4 (count=1)
Worker 1 processing job 4 (remaining=0)
Dispatcher added job 5 (count=1)
Worker 0 processing job 5 (remaining=0)
Dispatcher added job 6 (count=1)
Worker 1 processing job 6 (remaining=0)
Dispatcher added job 7 (count=1)
Worker 2 processing job 7 (remaining=0)
Dispatcher added job 8 (count=1)
Worker 1 processing job 8 (remaining=0)
Dispatcher added job 9 (count=1)
Worker 2 processing job 9 (remaining=0)
Dispatcher added job 10 (count=1)
Worker 1 processing job 10 (remaining=0)
All jobs dispatched. Exiting...
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA
10$

```

Observation:

Using condition variables removes CPU-wasting busy-waiting and synchronizes dispatcher-worker execution efficiently.

